

resiDNA_workflows

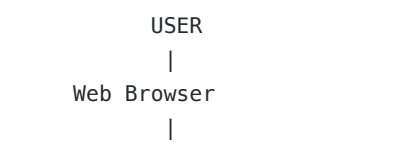
Residual DNA qPCR Analysis — Colab Notebook and Local Streamlit App

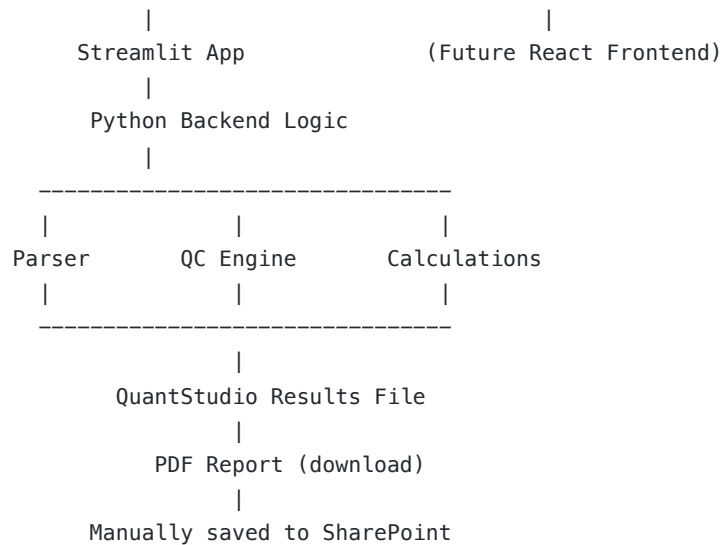
Project Organization

```
resiDNA_workflows/
|
|— app/
|   |— app.py                # Phase II Streamlit app – local web UI, mirrors
the notebook
|   |— audit_log.py          # opt-in audit log (RESIDNA_AUDIT_LOG) – app.py-
only, see Data Privacy below
|   |— requirements.txt      # dependencies for the Streamlit app
|— .streamlit/
|   |— config.toml           # light theme + wide layout (must stay at repo
root)
|
|— Notebooks/
|   |— 01_Phase I.ipynb      # Phase I notebook – import through PDF report
generation
|
|— Data/                      # local QuantStudio exports – real (scrubbed)
company data, not committed
|
|— Scripts/
|   |— qpcr.py               # shared classification/suitability logic –
imported by both the notebook and app.py
|   |— plotting.py           # shared style_table() – imported by both the
notebook and app.py
|— Figures/                  # reserved – future shared modules
|— Results/                  # generated PDF reports
|— Logs/                     # audit.log, only if RESIDNA_AUDIT_LOG is enabled
– gitignored, not committed
|— README.md
|— IT_Deployment_Guide.md    # IT handoff: on-prem / Azure + SharePoint hosting
paths (Option A decided)
  |— IT_Setup_Guide_OptionA.md # from-scratch runbook IT executes: commands,
service files, config templates
```

Application Architecture (In Progress)

Design of a local-first architecture, transitioning to IT-hosted per [IT_Deployment_Guide.md](#).





Development Road Map

Phase I - Colab Scientific Prototype (.ipynb)

Prove that the qPCR analysis logic works correctly.

1. ☒ Load QuantStudio file
2. ☒ QuantStudio Results Parser
3. ☒ Data Classification Engine
4. ☒ System Suitability Engine
5. ☒ Sample Result Processing
6. ☒ Create Prototype Graphs
7. ☒ Validate against `Data/sample_data_001.xls` and `Data/sample_data_002.xls`
8. ☒ PDF Report generation

Phase II - Convert Prototype into a Local Application (<http://localhost:8501>)

Create a simple website you run on your own computer using Local Streamlit App.

1. ☒ Upload Page
2. ☒ Backend Processing
3. ☒ Results Dashboard

Phase III - Add Criteria Management — **Done**

Stop changing code every time acceptance criteria change. Notebook: update thresholds directly in the `#@param` cell. App: an **Edit Acceptance Criteria** popup (session-only overrides, no database) — see Streamlit App section below.

Phase IV - Improve Reporting and Visualization — **Not needed**

Current interactive Plotly charts and the PDF report already meet the bar for a professional analytical tool; dropped from the roadmap.

Phase V - Hand Off to Company IT — **Next**

Move from prototype to maintainable, IT-hosted software. Pin dependencies, organize into a professional VS Code project, and transition hosting to company-controlled infrastructure per [IT Deployment Guide.md](#) (on-prem/local server or Azure + SharePoint embed) with Entra ID SSO.

Phase VI - (Maybe) Upgrade to Full Web Application

Only worth revisiting if IT's hosted deployment outgrows what Streamlit can offer (e.g. multi-user concurrency, custom auth flows beyond OIDC). Replace Streamlit with React Frontend and FastAPI Backend

Acceptance Criteria

STD (Standard Curve)

Check	Formula	Pass
R ²	instrument-computed	≥ 0.99
Ct %CV (triplicate)	$(Ct_SD / Ct_Mean) \times 100$	≤ 20%
Back-calculation bias	$ (Back-Calc Qty - Nominal Qty) / Nominal Qty \times 100$	≤ 25%
Amplification efficiency	$(10^{(-1/Slope)} - 1) \times 100$	90%–110%

ERC & PC (Controls)

Check	Formula	Pass
Quantity %CV (triplicate)	$(Qty_SD / Qty_Mean) \times 100$	≤ 20%
PC %Recovery	$(Dilution-Adjusted Qty / Reference-Adjusted Qty) \times 100$	80%–125%
ERC %Recovery	same formula	50%–150%

Sample LOQ / STD Range — replaces the old ICH-sigma LOQ formula. From the standard curve's own back-calculated values, take STD1 (highest nominal quantity) and STD6 (lowest):

Value	Source	Used for
LOQ (Quantity)	STD6 Back-Calc Mean	Sample STD Range / "below LOQ" floor
LOQ (Ct)	STD6 Ct Mean	NTC / NEC pass check
STD Range	[STD6 Back-Calc Mean, STD1 Back-Calc Mean]	Sample STD Range check

NTC / NEC — both pass per well if Ct is Undetermined **or** $Ct \geq L0Q_Ct$ (STD6's own measured Ct Mean above).

Sample Suitability — evaluated in this order, since each step feeds the next:

1. **Triplicate single-outlier exclusion:** if a dilution's full 3-well Quantity %CV fails 25%, `resolve_sample_replicates()` (`Scripts/qpcr.py`) drops the outlier well and re-checks the remaining 2; if they pass, that pair's mean becomes the dilution's Quantity %CV / Quantity Mean /

Total DNA / DNA per Protein ("Replicates Used: 2") instead of an outright fail. Only triggers on an already-failing triplicate — passing ones keep the instrument's reported figures as-is. Every check below uses this resolved `Quantity Mean` .

2. **Quantity %CV:** $(Qty_SD / Qty_Mean) \times 100 \leq 25\%$.
3. **STD Range:** the resolved `Quantity Mean` (pre-dilution-adjustment) must fall within the STD Range above; below it is `Out of Range` and is "below LOQ", above it is also `Out of Range` .
4. **Dilutional Linearity:** $\%Bias = (x_1 - x_2) / x_2 \times 100 \leq 20\%$, where x_2 is the `Dilution Adjusted` quantity of the sample's own dilution with the lowest combined Ct %CV + `Quantity %CV among dilutions that already pass both step 2 and step 3`, and x_1 is each dilution's `Dilution Adjusted` quantity. A sample with no dilution passing steps 2–3 has no valid reference, so every one of its dilutions gets `N/A` .
5. **Combined Suitability:** `Pass` only if steps 2–4 all pass; otherwise `Fail` (binary — no `N/A` , since an uninterpretable dilution isn't a soft middle ground). Only `Pass` dilutions get averaged into the final result.

Two tables: *Final Sample Results by Dilution* and *Final Sample Results — Averaged per Sample* (every sample is still reported, even one with zero passing dilutions — averaged across its passing dilutions if any exist, otherwise across all of them as a flagged fallback). In the averaged table: red highlight = the sample failed acceptance criteria (takes priority), green highlight = passed **and** `DNA per Protein` is below `SAMPLE_DNA_PER_PROTEIN_LIMIT` (15 ng/mg), gray + `- LOQ` suffix on `Total DNA / DNA per Protein` = the averaged `Quantity Mean` is below the LOQ (low confidence, footnoted). `Sample #` (e.g. `S1`) groups dilutions into the averaged table but isn't itself displayed in the by-dilution table — `Sample` (e.g. `S1 D1`) already carries that information.

Next Step — `compute_sample_status()` (`Scripts/qpcr.py`) turns the by-dilution Suitability breakdown into a next-action label per sample:

Next Step	Meaning
Reportable	At least one dilution cleanly passes all of steps 2–4 above (same condition as <code>Sample Passed</code>).
LOQ – Spike Test	<code>Quantity %CV</code> and <code>Dilutional Linearity</code> pass (<code>Linearity N/A</code> counts as non-blocking), but <code>STD Range</code> fails because the dilution's <code>Quantity Mean</code> is below <code>STD6</code> — below the assay's LOQ.
UL0Q – Adjust Dilution	Same as above, but <code>STD Range</code> fails because <code>Quantity Mean</code> is above <code>STD1</code> — above the curve's highest calibrated point.
Retest	Everything else: <code>STD Range</code> passes but <code>Quantity %CV</code> or <code>Linearity</code> fails, or multiple checks fail together.

Each dilution is classified individually first, then a sample with multiple dilutions reports whichever status ranks best across them (`Reportable` > `LOQ – Spike Test` / `UL0Q – Adjust Dilution` > `Retest`) — the same "any dilution passing is enough" rule already used for `Sample Passed` .

Sample Spike Recovery (Informational) — a follow-up confirmatory test, typically run on samples flagged `LOQ – Spike Test` above, to check whether a low/below-LOQ result reflects genuine low DNA or matrix inhibition suppressing recovery. A spiked dilution is identified by a `" S"` suffix on the sample name (e.g. `S1 D1 S`) and gets its own two checks:

Check	Formula	Pass
-------	---------	------

Quantity %CV (triplicate)	same single-outlier-exclusion resolution as unspiked dilutions (<code>resolve_sample_replicates()</code>)	$\leq 25\%$
% Recovery	instrument-computed: $100 \times (\text{Spiked Quantity Mean} - \text{Unspiked Quantity Mean}) / \text{Spike Input}$	80%– 125%

Reported in its own *Spike Recovery Suitability* table, placed below *Final Sample Results — Averaged per Sample*, green-highlighted where both checks pass. Deliberately informational only — it does not feed into or gate the corresponding unspiked dilution's own Suitability, Next Step, or whether it's averaged into the final result. Most runs have no spiked dilutions at all; spiking is an occasional follow-up test, not part of every dilution series, so this table (and section) is skipped entirely when there's nothing to report.

Criteria Verification

- **Output Verification:** all thresholds live in one Colab-form-editable cell (`#@param`) in the notebook, right after Data Classification. Values marked "instrument-computed" above are read forward from the QuantStudio export as-is (not recalculated) — the notebook's Output Verification cell checks they survive parsing unchanged. The one exception is Dilutional Linearity's `% Bias` , which is genuinely computed rather than read from the instrument.
- **Manual Golden-Reference Check:** a one-time sanity check for that computed `% Bias` value. Hand-calculate (or Excel-calculate) the expected `% Bias` for a few known dilutions against the reference-dilution formula, enter them into the notebook's `GOLDEN_LINEARITY_BIAS` dict, and re-run the cell — it flags any dilution where the notebook's computed value doesn't match your expected value within a ± 0.01 percentage-point tolerance (`GOLDEN_BIAS_TOLERANCE`).

PDF Report

Both the notebook and the Streamlit app generate the same formatted PDF summarizing a run — Run Info, System Suitability, Sample Suitability, Final Sample Results, and a Signatures block. In the notebook, this is the final `## PDF Report` section, using the `#@param` fields in the Report Info cell; it writes to `Results/Sample_Analysis_Report_<run_no>.pdf` and requires `reportlab` (auto-installs if missing). In the app, it's built the same way but in-memory and offered as a browser download instead.

- ☐ Logo support is wired up in both (falls back to a placeholder box until a file is present) — drop the real file at `app/assets/logo.png` .

Streamlit App (DEMO)

Demo deployment: <https://residna.streamlit.app> — hosted on Streamlit Community Cloud, no installation needed, demo use only. Access is restricted to authorized company viewers (see [Data Privacy & Security](#) before uploading real company data). See below to run it locally instead.

`app.py` is the Phase II local web app: upload a QuantStudio file and get the same System Suitability, Sample Suitability, and Final Sample Results sections as the notebook, without touching code, plus:

- **Sample ID / Sample Name** inputs — one row per Base Sample actually detected in the upload (simpler than the notebook's 8 fixed `#@param` slots, since the app can generate inputs from real data).
- **Edit Acceptance Criteria** — a popup for adjusting any threshold in the Acceptance Criteria section above (e.g. Sample Quantity %CV Max, PC/ERC %Recovery bounds) without touching code. See below.

- **Standard Curve graphs** (QC-controls overlay and per-sample overlay) and **Amplification Curves (ΔR_n vs Cycle)** — all interactive Plotly charts.
- A **Download PDF Report** button producing the same report described above.

Only the sections that map to the PDF report are shown — raw file preview, parse stats, and the classification table run silently in the background rather than cluttering the page.

Edit Acceptance Criteria

A button near the top opens a popup (`st.dialog`) with one box per criterion, grouped the same way as the Acceptance Criteria section above. Each box is empty, showing its current value grayed out as a placeholder — type a new value to override it, or leave it blank to keep the current one. A **Reset all to defaults** button clears every override at once.

Overrides are **session-only**: scoped to your browser tab, gone on reload. Trying a "what if this threshold were different" never silently changes results for anyone else or leaves a stale override for the next person on a shared deployment — deliberate, given the app currently has no login to attribute an override to a specific person. Whenever at least one criterion differs from its shipped default:

- An on-screen notice lists exactly which criteria changed, current default, and value used.
- The PDF report gets a **Modified Acceptance Criteria** section with the same table, placed right after Run Info.
- If the audit log is enabled (see below), the change is recorded there too — the only durable record that a report was generated with non-default thresholds, since the override itself is never saved anywhere.

Data Privacy & Security (DEMO Streamlit App)

In transit

- The Community Cloud link is served over HTTPS by default — traffic between the browser and Streamlit's servers is encrypted.

In the app

- `app.py` processes uploaded files entirely in memory (`st.file_uploader` → pandas parsing → Plotly charts / PDF generation). By default, nothing is written to disk or a database on the server — no `Results/` writes, no SQLite. Once a session ends or the app restarts, the uploaded file and its analysis are gone.
- The PDF report is streamed straight to the browser as a download; it isn't retained server-side.
- **Exception — audit log:** `app/audit_log.py` appends one record per generated PDF report (timestamp, uploaded filename, a SHA-256 hash of it, assay/run info, submitter/reviewer name, pass/fail, and any Edit Acceptance Criteria overrides active for that report — never the analytical results themselves) to `Logs/audit.log`, but only if the `RESIDNA_AUDIT_LOG` environment variable is set. It's unset (off) by default and on this demo deployment, so the "nothing persisted" behavior above still holds here — it's only enabled on the company-hosted deployment, where compliance requires it (see [IT Deployment Guide.md](#)).

Who can reach the app

- Access to the live link is restricted via Streamlit Community Cloud's viewer authentication: in the app's dashboard under **Settings → Sharing**, it is set to an allowlist of company email addresses — only people who sign in with one of those emails can open it.
- Residual risk worth knowing: even restricted, uploaded data still passes through Streamlit's shared Community Cloud infrastructure, which is intended for demos/public apps rather than as a

substitute for company-controlled hosting. For the most sensitive runs, run the app locally instead (see below) so data never leaves your machine or network.

For moving to a fully company-hosted deployment (on-prem/local server or Azure + SharePoint embed), see [IT_Deployment_Guide.md](#).

Running the Notebook Locally

The notebook auto-detects Colab vs. local execution (`IN_COLAB` flag) — outside Colab it skips `git clone / pip install klib / google.colab` imports and resolves paths relative to the repo instead of `/content/` .

```
python -m venv .venv
.venv\Scripts\pip install pandas numpy openpyxl xlrd klib ipykernel plotly reportlab
.venv\Scripts\python -m ipykernel install --user --name residDNA-venv --display-name
"residDNA (.venv)"
```

Then select the **residDNA (.venv)** kernel in VS Code / Jupyter and run all cells.

Running the Streamlit App Locally

Run these from the repo root — `.streamlit/config.toml` only applies when Streamlit is launched from the directory that contains it.

```
python -m venv .venv
.venv\Scripts\pip install -r app\requirements.txt
.venv\Scripts\streamlit run app\app.py
```

Opens at `http://localhost:8501` — only reachable while that command is still running in a terminal; closing it (or restarting your computer) stops the app until you run it again.

The audit log (see Data Privacy above) is off by default locally too. To try it: set `RESIDNA_AUDIT_LOG=1` before launching (`$env:RESIDNA_AUDIT_LOG="1"` in PowerShell), generate a report, then check `Logs/audit.log` .

Running Tests

```
python -m venv .venv
.venv\Scripts\pip install -r app\requirements.txt -r tests\requirements.txt
.venv\Scripts\pytest tests -v
```

Five suites, all under `tests/` :

- `test_qpcr.py` — unit tests for every pure function in `Scripts/qpcr.py` , independent of any data file.
- `test_app_golden.py` — runs the real `app/app.py` (headlessly, via a fake `streamlit` module in `tests/streamlit_stub.py`) against each file in `Data/` and checks `final_results / reportable_results` against known-good values, so a future change that silently shifts a Sample Suitability result gets caught automatically. `Data/*.xls / .xlsx` are real (scrubbed) company data and aren't committed to the repo (see Data Privacy above and git

history), so these tests **skip**, not fail, wherever those files aren't present — a fresh clone or CI run will show them skipped, which is expected.

- `test_audit_log.py` — unit tests for `app/audit_log.py`, including that it's a no-op unless `RESIDNA_AUDIT_LOG` is set and that raw analytical data never ends up in a log record.
- `test_app_criteria.py` — runs `app/app.py` with `st.session_state` seeded to simulate Edit Acceptance Criteria overrides, checking the override is applied, reported on-screen, in the PDF, and in the audit log — and that leaving everything at its default reports nothing.
- `test_sample_name_normalization.py` — confirms the whitespace-normalization step (Sample Name/Task, applied right after parsing) collapses analyst typos correctly and doesn't corrupt real NaN, including a "before/after" pair proving it fixes a genuine spike/unspiked Dilution Factor lookup failure, not just a hypothetical one.

Backlog — Recommended Cleanup

- ☐ Notebook outputs (styled tables, the `klib` plot) are committed inline, which makes diffs large. Consider `nbstripout` or committing a rendered HTML export alongside a stripped notebook.
- ☐ `Scripts/qpcr.py` / `Scripts/plotting.py` now hold the shared, self-contained logic (`style_table`, `classify_sample`, `recovery_bounds`, `combined_suitability`, the LOD sigma-Ct formula, the single-outlier triplicate resolution (`resolve_sample_replicates`), the Dilutional Linearity %Bias calculation, the Sample LOQ/STD Range derivation, STD Range suitability, the averaged-results fallback aggregation, the per-sample Next Step classification, and the Spike Recovery precision/recovery check) that both the notebook and `app.py` import. Deliberately left duplicated: the parsing/cleaning steps and suitability-table assembly, since the notebook keeps those inline, cell-by-cell, on purpose — that step-by-step visibility is Phase I's whole point (see Road Map). Worth revisiting only if that tradeoff stops being worth it.